

System call per la gestione dei processi

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

Informazioni preliminari

- ❑ Obiettivo: presentazione delle principali system call per la gestione dei processi*;
- ❑ **NOTA:** si consiglia di verificare l'installazione ed il funzionamento di **gcc** e **vi**.

* **Riferimenti:** Cap. 10 par. 10.3.1 (descrive anche *pipe* e *segnali*)
par. 10.3.2

Process identifier

- Ogni processo ha un **identificatore di processo** (o **process identifier**, **PID**), intero, tra 0 e un *massimo** assegnato dal kernel all'atto della sua creazione;
- È possibile ottenere i PID tramite le chiamate:

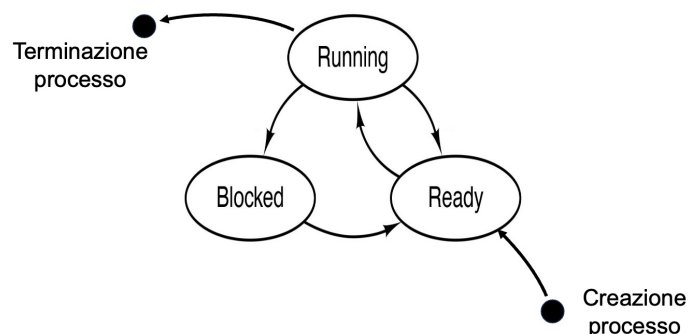
```
. int getpid()           //PID del processo  
. int getppid()          //PID del processo padre
```

*maximum PID Linux 64 bit:

```
$cat /proc/sys/kernel/pid_max  
4194304
```

Sleep

- Blocca un processo per un certo numero di **secondi** (transizione "*Running*" → "*Blocked*").
 - . `sleep(int sec)`



Fork

- ❑ La creazione di nuovi processi è gestita mediante la chiamata di sistema:

`int fork(void)`

- ❑ Crea una *copia esatta* del processo originale:
 - alla copia, il processo originale diventa il processo **genitore** (o **padre**); il nuovo processo è detto **figlio**;
 - il processo figlio è una copia del processo padre.

Fork*

studente@opsys:~\$ man fork

```

FORK(2)                                                    Linux Programmer's Manual          FORK(2)

NAME
    fork - create a child process

SYNOPSIS
    #include <sys/types.h>
    #include <unistd.h>

    pid_t fork(void);

DESCRIPTION
    fork() creates a new process by duplicating the calling process. The new process is referred to as the child process. The calling process is referred to as the parent process.

    The child process and the parent process run in separate memory spaces. At the time of fork() both memory spaces have the same content. Memory writes, file mappings (mmap(2)), and unmappings (munmap(2)) performed by one of the processes do not affect the other.

    The child process is an exact duplicate of the parent process except for the following points:

    * The child has its own unique process ID, and this PID does not match the ID of any existing process group (setpgid(2)) or session.

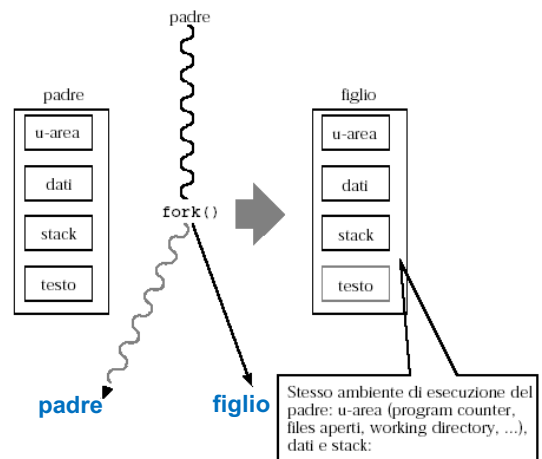
    * The child's parent process ID is the same as the parent's process ID.
```

...

* Esempio di voce di manuale per fork

Fork

- Le aree dati globali, stack, heap e u-area* sono copiate da padre e figlio, pertanto:
 - il figlio avrà una copia privata delle variabili, dei registri, e punta agli stessi file aperti...
- Attenzione!** Padre e figlio sono **isolati**: le variabili hanno valori identici *al momento* della chiamata `fork`, ma le successive modifiche di uno, non saranno visibili all'altro.



*u-area: per ogni processo, il SO mantiene una **user area** (u-area). La u-area contiene informazioni specifiche per il processo (per es., file aperti, directory corrente, etc.).

Fork

- Siccome è copiato anche il PC, entrambi i processi ripartono dall'**istruzione successiva** alla `fork`.
 - `fork` restituisce: (a) uno **0** (zero), al *figlio*, (b) un valore **≠0**, al processo *padre*.
 - è possibile differenziare il codice del padre da quello del figlio utilizzando il valore, intero, restituito dalla `fork`.

```
pid = fork( );           /* se la fork riesce, pid > 0 nel genitore */
if (pid < 0) {           /* fork fallita (per es., la memoria o qualche tabella è piena) */

} else if (pid > 0) {    /* codice genitore qui. */

} else {                /* codice figlio qui. */

}
```

Fork (implementazione pratica)

- La copia è costosa. Implementazioni pratiche:
 - l'area testo (che non è modificabile) è tipicamente condivisa tra padre e figlio in molte implementazioni UNIX;
 - condivisione in modalità **copy-on-write**:
 - inizialmente il figlio condivide la memoria del padre, configurata come read-only.
 - se uno dei due desidera modificarne una parte, il kernel provvede esplicitamente ad effettuare una copia di tale parte di memoria.

Wait ed exit

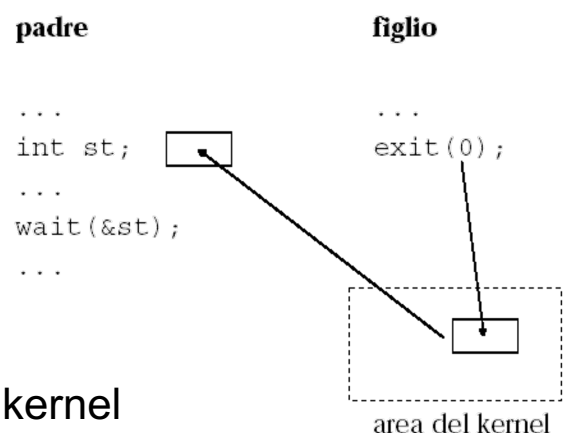
- wait consente al **processo padre** di raccogliere lo stato di terminazione dei figli

- **stato** contiene il valore passato dal figlio alla system call `exit`.

```
int wait(int* stato)
```

```
int waitpid(pid_t pid, int* stato,  
            int options)
```

- se i figli *non* sono ancora terminati, il kernel **sospende** il processo padre finché *uno* dei figli (`wait`) o uno *specifico figlio* (`waitpid`) non è terminato.



Casi tipici

- Un processo (figlio) termina **prima** che il proprio padre abbia avuto modo di attenderne la fine:
 - il processo terminato è definito **zombie**; il processo, anche se terminato, ha ancora un PID e il PCB, necessari affinché il padre possa raccogliergli il valore di uscita.
- Il processo padre termina **prima** dei suoi processi figli:
 - i processi figli sono detti **orfani**;
 - i processi orfani sono adottati da `init` (`systemd`) e, quindi, il pid del padre diventa 1.

Esempio con fork e wait

```
printf("PID = %d; p_PID = %d: Hello (before fork) \n", getpid(), getppid() );

int pid = fork();

if ( pid < 0 ){
    printf("Errore (fork failed)");
}
else if ( pid > 0 ){    // --- CODICE DEL PADRE ---

    int status=0;
    int p =0;

    for ( int i=0; i<5; i++ ){
        sleep(2);
        printf("PID = %d; p_PID = %d: Hello # %d\n", getpid(), getppid(), i );
    }

    printf ( "In attesa...\n" );
    p = wait ( &status );
    printf ("Figlio %d terminato con stato %d\n", p, status);

}
else{    // --- CODICE DEL FIGLIO ---

    for ( int i=0; i<10; i++ ){
        sleep(2);
        printf("\t\t\t\t\tPID = %d; p_PID = %d: Hello # %d\n", getpid(), getppid(), i );
    }

}
```

Esecuzione

//processo padre

```
PID = 34528; p_PID = 33997: Hello (before fork)
PID = 34528; p_PID = 33997: Hello # 0

PID = 34528; p_PID = 33997: Hello # 1

PID = 34528; p_PID = 33997: Hello # 2

PID = 34528; p_PID = 33997: Hello # 3

PID = 34528; p_PID = 33997: Hello # 4
In attesa...
```

//processo figlio

```
PID = 34529; p_PID = 34528: Hello # 0

PID = 34529; p_PID = 34528: Hello # 1

PID = 34529; p_PID = 34528: Hello # 2

PID = 34529; p_PID = 34528: Hello # 3

PID = 34529; p_PID = 34528: Hello # 4
PID = 34529; p_PID = 34528: Hello # 5
PID = 34529; p_PID = 34528: Hello # 6
PID = 34529; p_PID = 34528: Hello # 7
PID = 34529; p_PID = 34528: Hello # 8
PID = 34529; p_PID = 34528: Hello # 9
```

Figlio 34529 terminato con stato 0

Famiglia di funzioni exec

- L'**exec** effettua una **sostituzione** di codice:
 - consente ad un processo di eseguire un programma differente;
 - il **nuovo programma** viene eseguito nel contesto del processo chiamante (il pid *non cambia*).

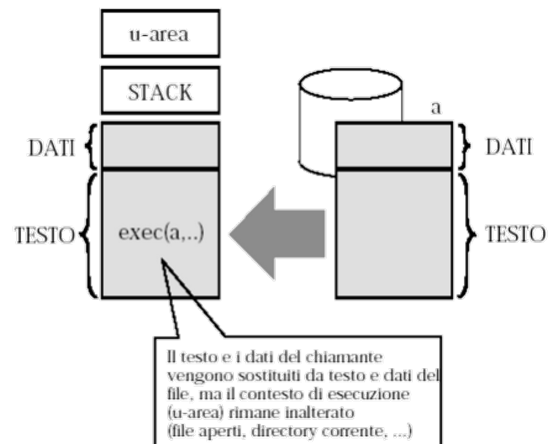
NOTA: *non esiste* una chiamata di sistema exec, ma bisogna ricorrere ad una delle procedure in tabella.

* queste varianti di exec cercano il file/comando nel PATH

Member name	Using pathname	Using filename	Argument List	Argument Array	Original ENV	Provided ENV
execl()	YES		YES		YES	
execlp()		YES*	YES		YES	
execle()	YES		YES			YES
execv()	YES			YES	YES	
execvp()		YES*		YES	YES	
execve()	YES			YES		YES

Famiglia di funzioni exec

- L'invocazione di exec:
 - fa ritorno al chiamante in caso di errore, altrimenti il controllo passa al **nuovo programma**.
- Un processo dopo l'exec:
 - ha codice, dati globali, stack e heap **nuovi**;
 - mantiene user area (a parte PC e informazioni legate al codice).



Esempio con exec

```
#include <stdio.h>
#include <unistd.h>

int main (){
    for ( char c='a'; c<='l'; c++){
        sleep(1);
        printf("PID = %d; p_PID = %d: print char %c\n", getpid(), getppid(), c);

        if ( c=='e' ) {
            if ( execl("/home/studente/esempi/initialprog", "initialprog", NULL) < 0 )
                printf("Errore (exec failed)");
        }
    }

    return 0;
}
```

//esecuzione

```
PID = 34706; p_PID = 33997: print char a
PID = 34706; p_PID = 33997: print char b
PID = 34706; p_PID = 33997: print char c
PID = 34706; p_PID = 33997: print char d
PID = 34706; p_PID = 33997: print char e
PID = 34706; p_PID = 33997: Hello # 0
PID = 34706; p_PID = 33997: Hello # 1
PID = 34706; p_PID = 33997: Hello # 2
...
```